

Практикум 3. Множественное выравнивание.

1. Какие последовательности использовали для анализа (тип биополимера, организм, ген) и какими программами пользовались? Укажите версии программ.

Kluyveromyces lactis (budding yeasts) <https://www.ncbi.nlm.nih.gov/Taxonomy/Browser/wwwtax.cgi?id=28985> - это дрожжи, способные усваивать лактозу и превращать ее в молочную кислоту (и поэтому используемые для генетических исследований и промышленных применений)

- SUP35_10seqs.fa - нуклеотидные последовательности гена, кодирующего фактор терминации/трансляции для 10 разных видов дрожжей
- SUP35_250seqs.fa - нуклеотидные последовательности гена, кодирующего фактор терминации/трансляции для 250 разных видов дрожжей
- SUP35_10seqs.fa - файл, в котором что-то не так
- SUP35_2addseqs.fa - 2 последовательности, которые надо добавить к 250 выровненным последовательностям из SUP35_250seqs.fa

```
In [1]: import pandas as pd
```

```
In [2]: #многие биологи считают его алгоритмом по умолчанию
! clustalw
```

```
*****
***** CLUSTAL 2.1 Multiple Sequence Alignments *****
*****
```

1. Sequence Input From Disc
2. Multiple Alignments
3. Profile / Structure Alignments
4. Phylogenetic trees

- S. Execute a system command
- H. HELP
- X. EXIT (leave program)

Your choice: ^C

```
In [3]: ! muscle -version
```

MUSCLE v3.8.1551 by Robert C. Edgar

```
In [4]: ! mafft --version
```

```
In [5]: ! kalign
```

Kalign version 2.04, Copyright (C) 2004, 2005, 2006 Timo Lassmann

Kalign is free software. You can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation.

reading from STDIN: found no sequences.

Usage: kalign2 [INFILE] [OUTFILE] [OPTIONS]

Options:

-s,	-gapopen -gap_open -gpo	Gap open penalty
-e,	-gapextension -gap_ext -gpe	Gap extension penalty
-t,	-terminal_gap_extension_penalty -tgpe	Terminal gap penalties
-m,	-matrix_bonus -bonus	A constant added to the substitution matrix.
-c,	-sort	The order in which the sequences appear in the output alignment. <input, tree, gaps.>
-g,	-feature -same_feature_score -diff_feature_score	Selects feature mode and specifies which features are to be used: e.g. all, maxplp, STRUCT, PFAM-A... Score for aligning same features Penalty for aligning different features
-d,	-distance	Distance method. <wu,pair>
-b,	-guide-tree -tree	Guide tree method. <nj,upgma>
-z,	-zcutoff	Parameter used in the wu-manber based distance calculation
-i,	-input -infile -in	The input file.

```

-o,      -output      The output file.
         -outfile
         -out

-a,      -gap_inc      Parameter increases gap penalties depending on the number of existing gaps

-f,      -format      The output format:
                     <fasta, msf, aln, clu, macsim>

-q,      -quiet        Print nothing to STDERR.
                     Read nothing from STDIN

Examples:

Using pipes:
    kalign2 [OPTIONS] < [INFILE]   > [OUTFILE]
    more [INFILE] | kalign2 [OPTIONS] > [OUTFILE]

Relaxed gap penalties:
    kalign2 -gpo 60 -gpe 9 -tgpe 0 -bonus 0 < [INFILE]   > [OUTFILE]

Feature alignment with pairwise alignment based distance method and NJ guide tree:
    kalign2 -in test.xml -distance pair -tree nj -sort gaps -feature STRUCT -format macsim -out test.macsim

```

WARNING: No sequences found.

In [6]: `! t_coffee -version`

PROGRAM: T-COFFEE Version_12.00.7fb08c2 (2018-12-11 09:27:12 - Revision 7fb08c2 - Build 211)

In [7]: `! prank -version`

This is PRANK v.170427.
 Checking if updates are available at <http://http://code.google.com/p/prank-msa>.

Found updates. Changes in the more recent versions:

```

<!DOCTYPE html>
<html lang=en>
  <meta charset=utf-8>
  <meta name=viewport content="initial-scale=1, minimum-scale=1, width=device-width">
  <title>Error 404 (Not Found)!!1</title>
  <style>
    *{margin:0;padding:0}html,code{font:15px/22px arial,sans-serif}html{background:#fff;color:#222;padding:15px}body{margin:7% auto 0;max-width:390px;min-height:180px;padding:30px 0 15px}* > body{background:url(//www.google.com/images/errors/robot.png) 100% 5px no-repeat;padding-right:205px}p{margin:11px 0 22px;overflow:hidden}ins{color:#777;text-decoration:none}a img{border:0}@media screen and (max-width:772px){body{background:none;margin-top:0;max-width:none;padding-right:0}}#logo{background:url(//www.google.com/images/branding/googlelogo/1x/googlelogo_color_150x54dp.png) no-repeat;margin-left:-5px}@media only screen and (min-resolution:192dpi){#logo{background:url(//www.google.com/images/branding/g

```

```
ooglelogo/2x/googlelogo_color_150x54dp.png) no-repeat 0% 0%/100% 100%;-moz-border-image:url(//www.google.com/images/branding/googlelogo/2x/googlelogo_color_150x54dp.png) 0}}@media only screen and (-webkit-min-device-pixel-ratio:2){#logo{background:url(//www.google.com/images/branding/googlelogo/2x/googlelogo_color_150x54dp.png) no-repeat;-webkit-background-size:100% 100%}}#logo{display:inline-block;height:54px;width:150px}
</style>
<a href=//www.google.com/><span id=logo aria-label=Google></span></a>
<p><b>404.</b> <ins>That's an error.</ins>
<p>The requested URL <code>/git/VERSION_HISTORY</code> was not found on this server. <ins>That's all we know.</ins>
```

2. Код для запуска 6 возможных алгоритмов выравнивания (clustalw, muscle, mafft, kalign, tcoffee, prank) для 10 последовательностей ДНК (SUP35_10seqs.fa) + вариации параметров, если они были. Если всё запускали онлайн — ссылки на страницы, можно приложить скриншоты, если это кажется осмысленным.

In [28]:

```
%%capture

#сохраняет в собственный формат (CLUSTAL-Alignment file - .aln). Большинство программ его читает, но все же лучше в fasta, поэтому надо пропис
! clustalw -INFILE=SUP35_10seqs.fa -OUTPUT=FASTA -OUTFILE=SUP35_10seqs.clustalw.fa

#адекватное название параметров :)
! muscle -in SUP35_10seqs.fa -out SUP35_10seqs_muscle.fa

#считает себя ускоренным
#--auto - определяет оптимальные параметры (скорость/успешное вычисление)
! mafft --auto SUP35_10seqs.fa >SUP35_10seqs_mafft.fa

#очень быстрый
! kalign < SUP35_10seqs.fa > SUP35_10seqs_kalign.fa

#долгий, запускает внутри много алгоритмов, поэтому, по мнению разработчиков, работает прекрасно
! t_coffee -infile=SUP35_10seqs.fa -output=fasta -outfile=SUP35_10seqs_tcoffee.fa

! prank -d=SUP35_10seqs.fa -o=SUP35_10seqs_prank.fa
```

3. Сравнительная таблица со временем работы* и комментариями по поводу качества выравнивания ДНК для указанных выше алгоритмов**. Какой алгоритм лучше использовать?

Необходимый минимум: время выполнения + длина выравнивания или графическое представление (например, как в UGENE). Бонус за более глубокий анализ.

- Время выполнения при запуске в терминале можно посмотреть с помощью команды time.
- Если построение выравнивания требует неразумно большого времени, допустима запись вроде "> 15 минут, надоело" :)

Комментарии:

- Для оценки качества обычно используют референсные выравнивания (в спец.базах данных). Но чаще люди оценивают "на глаз", что больше нравится. Нет какого-то принятого стандарта.
- Среди выровненных последовательностей в целом лучше более короткое выравнивание.
- Score разные тулы считают по-разному, поэтому сравнивать по ним не получится
- Можно оценивать по методу максимального правдоподобия при построении деревьев

In [82]:

```
%%capture

%%script bash

echo -e 'tool\ttime,m:s\tlength' > time.txt

echo -n -e 'clustalw\t' >> time.txt
/usr/bin/time -ao time.txt -f "%E" clustalw -INFILE=SUP35_10seqs.fa -OUTPUT=FASTA -OUTFILE=SUP35_10seqs_clustalw.fa
awk 'NR > 1 { print prev } { prev=$0 } END { ORS=""; print }' time.txt > time10.txt
echo -e -n '\t' >> time10.txt
awk '/^>/{l=0; next}{l+=length($0)}END{print l}' SUP35_10seqs_clustalw.fa >> time10.txt
echo '' >> time10.txt

echo -n -e 'muscle\t' > time.txt
/usr/bin/time -ao time.txt -f "%E" muscle -in SUP35_10seqs.fa -out SUP35_10seqs_muscle.fa
awk 'NR > 1 { print prev } { prev=$0 } END { ORS=""; print }' time.txt >> time10.txt
echo -e -n '\t' >> time10.txt
awk '/^>/{l=0; next}{l+=length($0)}END{print l}' SUP35_10seqs_muscle.fa >> time10.txt
echo '' >> time10.txt

echo -n -e 'mafft\t' > time.txt
/usr/bin/time -ao time.txt -f "%E" mafft --auto SUP35_10seqs.fa > SUP35_10seqs_mafft.fa
awk 'NR > 1 { print prev } { prev=$0 } END { ORS=""; print }' time.txt >> time10.txt
echo -e -n '\t' >> time10.txt
awk '/^>/{l=0; next}{l+=length($0)}END{print l}' SUP35_10seqs_mafft.fa >> time10.txt
echo '' >> time10.txt

echo -n -e 'kalign\t' > time.txt
/usr/bin/time -ao time.txt -f "%E" kalign < SUP35_10seqs.fa > SUP35_10seqs_kalign.fa
awk 'NR > 1 { print prev } { prev=$0 } END { ORS=""; print }' time.txt >> time10.txt
echo -e -n '\t' >> time10.txt
awk '/^>/{l=0; next}{l+=length($0)}END{print l}' SUP35_10seqs_kalign.fa >> time10.txt
echo '' >> time10.txt

echo -n -e 't_coffee\t' > time.txt
/usr/bin/time -ao time.txt -f "%E" t_coffee -infile=SUP35_10seqs.fa -output=fasta -outfile=SUP35_10seqs_tcoffee.fa
awk 'NR > 1 { print prev } { prev=$0 } END { ORS=""; print }' time.txt >> time10.txt
echo -e -n '\t' >> time10.txt
awk '/^>/{l=0; next}{l+=length($0)}END{print l}' SUP35_10seqs_tcoffee.fa >> time10.txt
echo '' >> time10.txt
```

```
echo -n -e 'prank\t' > time.txt
/usr/bin/time -ao time.txt -f "%E" prank -d=SUP35_10seqs.fa -o=SUP35_10seqs_prank.fa
awk 'NR > 1 { print prev } { prev=$0 } END { ORS=""; print }' time.txt >> time10.txt
echo -e -n '\t' >> time10.txt
awk '/^>/{l=0; next}{l+=length($0)}END{print l}' SUP35_10seqs_prank.fa.best.fas >> time10.txt
echo '' >> time10.txt
```

In [83]: `pd.read_csv('time10.txt', sep = '\t')`

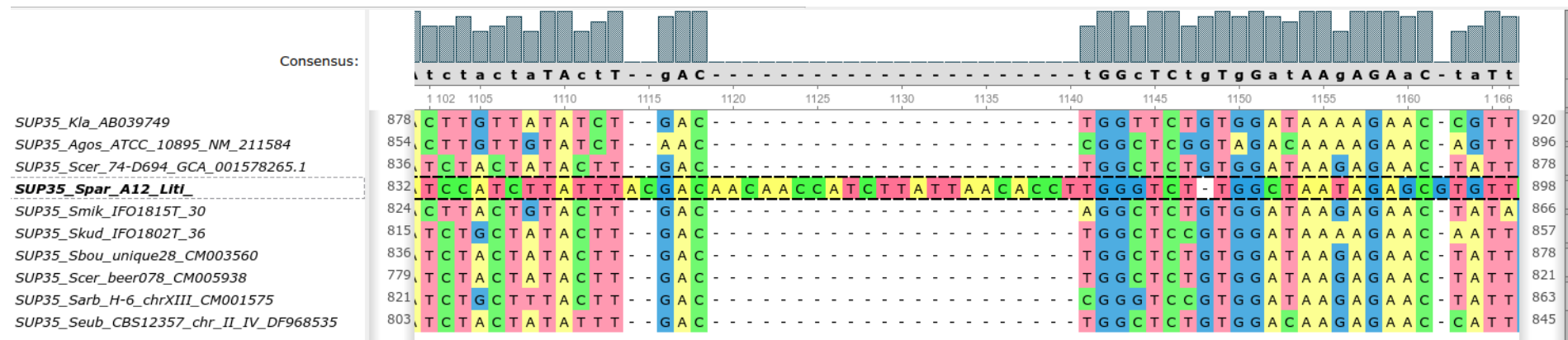
Out[83]:

	tool	time,m:s	length
0	clustalw	0:03.57	2148
1	muscle	0:02.54	2275
2	mafft	0:02.38	2166
3	kalign	0:00.20	2155
4	t_coffee	0:13.36	2210
5	prank	0:05.19	2366

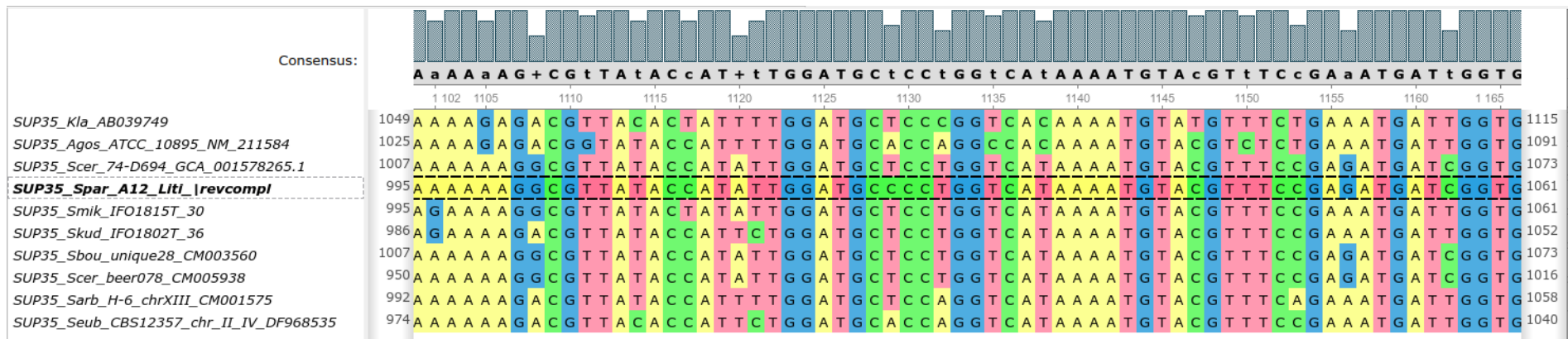
Видим, что у kalign минимальное время, а у clustalw минимальная длина (но 2148 и 2155 - это почти одно и то же)

4. Что не так с выравниванием SUP35_10seqs_strange_aln.fa и как это исправить?

Есть одна последовательность, непохожая на все остальные



Она reverse-complement к остальным цепям. Ее надо перевернуть и выровнять (например, kalign) заново.



Для нахождения таких штук можно строить деревья, и тогда они будут очень далеко от остальных.

5. Команды для запуска 6 возможных вариантов выравнивания (см. п. 2), но для 250 последовательностей ДНК.

```
In [ ]: %%capture

! clustalw -INFILE=SUP35_250seqs.fa OUTPUT=FASTA -OUTFILE=SUP35_250seqs.clustalw.fa
! muscle -in SUP35_250seqs.fa -out SUP35_250seqs_muscle.fa
! mafft --auto SUP35_250seqs.fa >SUP35_250seqs_mafft.fa
! kalign <SUP35_250seqs.fa >SUP35_250seqs_kalign.fa
! t_coffee -infile=SUP35_250seqs.fa -outfile=SUP35_250seqs_tcoffee.fa
! prank -d=SUP35_250seqs.fa -o=SUP35_250seqs_prank.fa
```

6. Сравнительная таблица со временем работы и комментариями по поводу качества выравнивания 250 последовательностей ДНК (SUP35_250seqs.fa). Изменился ли наш выбор алгоритма?

```
In [ ]: %%capture

%%script bash

echo -e 'tool\ttime,m:s\tlength' > time.txt

echo -n -e 'clustalw\t' >> time.txt
/usr/bin/time -ao time.txt -f "%E" clustalw -INFILE=SUP35_250seqs.fa -OUTPUT=FASTA -OUTFILE=SUP35_250seqs_clustalw.fa
awk 'NR > 1 { print prev } { prev=$0 } END { ORS=""; print }' time.txt > time250.txt
echo -e -n '\t' >> time250.txt
awk '/^>/{l=0; next}{l+=length($0)}END{print l}' SUP35_250seqs_clustalw.fa >> time250.txt
echo '' >> time250.txt
```

```

echo -n -e 'muscle\t' > time.txt
/usr/bin/time -ao time.txt -f "%E" muscle -in SUP35_250seqs.fa -out SUP35_250seqs_muscle.fa
awk 'NR > 1 { print prev } { prev=$0 } END { ORS=""; print }' time.txt >> time250.txt
echo -e -n '\t' >> time250.txt
awk '/^>/{l=0; next}{l+=length($0)}END{print l}' SUP35_250seqs_muscle.fa >> time250.txt
echo '' >> time250.txt

echo -n -e 'mafft\t' > time.txt
/usr/bin/time -ao time.txt -f "%E" mafft --auto SUP35_250seqs.fa > SUP35_250seqs_mafft.fa
awk 'NR > 1 { print prev } { prev=$0 } END { ORS=""; print }' time.txt >> time250.txt
echo -e -n '\t' >> time250.txt
awk '/^>/{l=0; next}{l+=length($0)}END{print l}' SUP35_250seqs_mafft.fa >> time250.txt
echo '' >> time250.txt

echo -n -e 'kalign\t' > time.txt
/usr/bin/time -ao time.txt -f "%E" kalign < SUP35_250seqs.fa > SUP35_250seqs_kalign.fa
awk 'NR > 1 { print prev } { prev=$0 } END { ORS=""; print }' time.txt >> time250.txt
echo -e -n '\t' >> time250.txt
awk '/^>/{l=0; next}{l+=length($0)}END{print l}' SUP35_250seqs_kalign.fa >> time250.txt
echo '' >> time250.txt

echo -n -e 't_coffee\t' > time.txt
/usr/bin/time -ao time.txt -f "%E" t_coffee -infile=SUP35_250seqs.fa -output=fasta -outfile=SUP35_250seqs_tcoffee.fa
awk 'NR > 1 { print prev } { prev=$0 } END { ORS=""; print }' time.txt >> time250.txt
echo -e -n '\t' >> time250.txt
awk '/^>/{l=0; next}{l+=length($0)}END{print l}' SUP35_250seqs_tcoffee.fa >> time250.txt
echo '' >> time250.txt

echo -n -e 'prank\t' > time.txt
/usr/bin/time -ao time.txt -f "%E" prank -d=SUP35_250seqs.fa -o=SUP35_250seqs_prank.fa
awk 'NR > 1 { print prev } { prev=$0 } END { ORS=""; print }' time.txt >> time250.txt
echo -e -n '\t' >> time250.txt
awk '/^>/{l=0; next}{l+=length($0)}END{print l}' SUP35_250seqs_prank.fa.best.fas >> time250.txt
echo '' >> time250.txt

```

коротко о t-coffee на локальном ноутбуке


```
>SUP35_K1a_AB039749_1
MSDQQNQDQGQGGQYNQYNQYGGYNQYYNQGGYQGYNGQQGAPQGYQAYQAYGQQPQGGAY
QGYNPQQAQGYQPYQGYNAQQQGYNAQQGGHNNNNYNNKNNKNSYNNYNKQGYQGAQGYN
AQQPTGYAAPAQSSSQGMTLKDFQNNQGGSTNAAKPKPKLKLASSSGIKLVGAKKPVAPKT
EKTDESKEATKTTDDNEEAQSELPKIDDLKISEAEKPKTKENTPSADDTSEKTTSAKAD
TSTGGANSVDALIKEQEDEVDEEVVKDMFGGKDHVSIIFMGHVDAGKSTMGGNLLYLTGS
VDKRTVEKYEREAKEAGRQGWYLSWMDTNKEERNNDGKTIEVGRAYFETEKRRYTILDAP
GHKMYVSEMIGGASQADIGILVISARKGEYETGFEGGQTREHALLAKTQGVNKMIVVIN
KMDDPTVGWDKERYDHCVGNLTNFLKAVGYNVKEDVIFMPVSGYTGAGLKERVDPKDCPW
YTGPSLLEYLDNMKTDRHINAPFMLPIASKMKMDMGTVVEGKIESGHIRKGNQTLLMPNR
```

```
In [37]: #ищет ORF - open reading frame - все между старт и стоп кодоном
! getorf -sequence SUP35_10seqs.fa -outseq SUP35_10seqs.g.faa -noreverse -minsize 500
```

Find and extract open reading frames (ORFs)

```
In [44]: #добавляет к названию информацию о начале и конце рамки
! head SUP35_10seqs.g.faa
```

```
>SUP35_K1a_AB039749_1 [1 - 2100]
MSDQQNQDQGQGGQYNQYNQYGGYNQYYNQGGYQGYNGQQGAPQGYQAYQAYGQQPQGGAY
QGYNPQQAQGYQPYQGYNAQQQGYNAQQGGHNNNNYNNKNNKNSYNNYNKQGYQGAQGYN
AQQPTGYAAPAQSSSQGMTLKDFQNNQGGSTNAAKPKPKLKLASSSGIKLVGAKKPVAPKT
EKTDESKEATKTTDDNEEAQSELPKIDDLKISEAEKPKTKENTPSADDTSEKTTSAKAD
TSTGGANSVDALIKEQEDEVDEEVVKDMFGGKDHVSIIFMGHVDAGKSTMGGNLLYLTGS
VDKRTVEKYEREAKEAGRQGWYLSWMDTNKEERNNDGKTIEVGRAYFETEKRRYTILDAP
GHKMYVSEMIGGASQADIGILVISARKGEYETGFEGGQTREHALLAKTQGVNKMIVVIN
KMDDPTVGWDKERYDHCVGNLTNFLKAVGYNVKEDVIFMPVSGYTGAGLKERVDPKDCPW
YTGPSLLEYLDNMKTDRHINAPFMLPIASKMKMDMGTVVEGKIESGHIRKGNQTLLMPNR
```

Проблема: Надо задавать minsize (минимальный размер рамки считывания), иначе находит очень много рамок считывания на каждую последовательность, большинство из которых неадекватные (если мы знаем, что ген должен быть какого-то не совсем маленького размера, то надо задать границу, тк чаще всего мы хотим получить одну лучшую рамку считывания)

```
In [45]: ! getorf -sequence SUP35_10seqs.fa -outseq SUP35_10seqs.g_bad.faa -noreverse
! head SUP35_10seqs.g_bad.faa
```

Find and extract open reading frames (ORFs)

```
>SUP35_K1a_AB039749_1 [3 - 53]
VRPTKSRPRARPRLQSV
>SUP35_K1a_AB039749_2 [57 - 278]
PIWPVQPVLQPTGLSRLQRPTRCSSRLPSISSLWTAASRSLPGLQPSTSSRLPTLPGLQC
SATRLQRSARRSQ
>SUP35_K1a_AB039749_3 [2 - 412]
CQTNKIKTKGKAKVTISITNMASTTTTNNRAIKATTANKVLLKATKHIKLMDSSLKEPT
RATTLNKLKATNLTRATMLSNKVTTLSKAVTTITTIKIIIIKTVTITIIISRVIKVLKDIM
HNSQPVTLQLHSLHPRV
>SUP35_K1a_AB039749_4 [360 - 449]
```

8. Команды для запуска 6 возможных вариантов выравнивания для 10 белковых последовательностей + вариации параметров, если они были.

Большинство алгоритмов сами определяют, дали им на вход белок или нуклеотид (по первым 40-100 буквам)

```
In [ ]: %%capture

%%bash

#clustalw надо сказать -TYPE=protein
clustalw -INFILE=SUP35_10seqs.g.faa -OUTFILE=SUP35_10seqs.clustalw.g.faa -OUTPUT=FASTA -TYPE=protein

#--force - чтобы перезаписать файл
clustalo --infile=SUP35_10seqs.g.faa --outfile=SUP35_10seqs.clustalo.g.faa --verbose --force
muscle -in SUP35_10seqs.g.faa -out SUP35_10seqs_muscle.g.faa
mafft --auto SUP35_10seqs.g.faa >SUP35_250seqs_mafft.g.faa
kalign <SUP35_10seqs.g.faa >SUP35_10seqs_kalign.g.faa
t_coffee -infile=SUP35_10seqs.g.faa -output=fasta -outfile=SUP35_10seqs_tcoffee.g.faa
prank -d=SUP35_10seqs.g.faa -o=SUP35_10seqs_prank.g.faa
```

9. Сравнительная таблица со временем работы и комментариями по поводу качества выравнивания белков. Какой алгоритм лучше использовать?

```
In [78]: %%capture

%%script bash

echo -e 'tool\ttime,m:s\tlength' > time.txt

echo -n -e 'clustalw\t' >> time.txt
/usr/bin/time -ao time.txt -f "%E" clustalw -INFILE=SUP35_10seqs.g.faa -OUTFILE=SUP35_10seqs.clustalw.g.faa -OUTPUT=FASTA -TYPE=protein

awk 'NR > 1 { print prev } { prev=$0 } END { ORS=""; print }' time.txt > time_p.txt
echo -e -n '\t' >> time_p.txt
awk '/^>/{l=0; next}{l+=length($0)}END{print l}' SUP35_10seqs.clustalw.g.faa >> time_p.txt
echo '' >> time_p.txt

echo -n -e 'clustalo\t' >> time.txt
/usr/bin/time -ao time.txt -f "%E" clustalo --infile=SUP35_10seqs.g.faa --outfile=SUP35_10seqs.clustalo.g.faa --verbose --force
awk 'NR > 1 { print prev } { prev=$0 } END { ORS=""; print }' time.txt > time_p.txt
echo -e -n '\t' >> time_p.txt
awk '/^>/{l=0; next}{l+=length($0)}END{print l}' SUP35_10seqs.clustalo.g.faa >> time_p.txt
echo '' >> time_p.txt
```

```

echo -n -e 'muscle\t' > time.txt
/usr/bin/time -ao time.txt -f "%E" muscle -in SUP35_10seqs.g.faa -out SUP35_10seqs_muscle.g.faa

awk 'NR > 1 { print prev } { prev=$0 } END { ORS=""; print }' time.txt >> time_p.txt
echo -e -n '\t' >> time_p.txt
awk '/^>/{l=0; next}{l+=length($0)}END{print l}' SUP35_10seqs_muscle.g.faa >> time_p.txt
echo '' >> time_p.txt

echo -n -e 'mafft\t' > time.txt
/usr/bin/time -ao time.txt -f "%E" mafft --auto SUP35_10seqs.g.faa >SUP35_250seqs_mafft.g.faa

awk 'NR > 1 { print prev } { prev=$0 } END { ORS=""; print }' time.txt >> time_p.txt
echo -e -n '\t' >> time_p.txt
awk '/^>/{l=0; next}{l+=length($0)}END{print l}' SUP35_250seqs_mafft.g.faa >> time_p.txt
echo '' >> time_p.txt

echo -n -e 'kalign\t' > time.txt
/usr/bin/time -ao time.txt -f "%E" kalign <SUP35_10seqs.g.faa >SUP35_10seqs_kalign.g.faa

awk 'NR > 1 { print prev } { prev=$0 } END { ORS=""; print }' time.txt >> time_p.txt
echo -e -n '\t' >> time_p.txt
awk '/^>/{l=0; next}{l+=length($0)}END{print l}' SUP35_10seqs_kalign.g.faa >> time_p.txt
echo '' >> time_p.txt

echo -n -e 't_coffee\t' > time.txt
/usr/bin/time -ao time.txt -f "%E" t_coffee -infile=SUP35_10seqs.g.faa -output=fasta -outfile=SUP35_10seqs_tcoffee.g.faa

awk 'NR > 1 { print prev } { prev=$0 } END { ORS=""; print }' time.txt >> time_p.txt
echo -e -n '\t' >> time_p.txt
awk '/^>/{l=0; next}{l+=length($0)}END{print l}' SUP35_10seqs_tcoffee.g.faa >> time_p.txt
echo '' >> time_p.txt

echo -n -e 'prank\t' > time.txt
/usr/bin/time -ao time.txt -f "%E" prank -d=SUP35_10seqs.g.faa -o=SUP35_10seqs_prank.g.faa
awk 'NR > 1 { print prev } { prev=$0 } END { ORS=""; print }' time.txt >> time_p.txt
echo -e -n '\t' >> time_p.txt
awk '/^>/{l=0; next}{l+=length($0)}END{print l}' SUP35_10seqs_prank.g.faa.best.fas >> time_p.txt
echo '' >> time_p.txt

```

In [81]: `pd.read_csv('time_p.txt', sep = '\t')`

Out[81]:

	tool	time,m:s	length
0	clustalw	0:00.40	719
1	clustalo	0:00.32	757

	tool	time,m:s	length
2	muscle	0:00.12	743
3	mafft	0:00.40	759
4	kalign	0:00.03	720
5	t_coffee	0:01.82	752
6	prank	0:07.21	791

По времени все быстро, выиграл, как обычно, kalign. Минимальная длина - у clustalw, у kalign примерно то же самое (719 и 720)

10. Как добавить к выравниванию 250 нуклеотидных последовательностей ещё две (SUP35_2addseqs.fsa), предварительно выровняв их, с помощью mafft и muscle?

1. Выравниваем добавляемые последовательности между собой
2. Добавляем к имеющемуся выравниванию

In [85]:

```
%%capture
%%script bash

muscle -in SUP35_2addseqs.fsa -out SUP35_2addseqs_muscle.fsa
muscle -profile -in1 SUP35_250seqs_muscle.fsa -in2 SUP35_2addseqs.fsa -out SUP35_252seqs_muscle.fsa
mafft --auto SUP35_2addseqs.fsa > SUP35_2addseqs_mafft.fsa
mafft --add SUP35_2addseqs_mafft.fsa SUP35_250seqs_mafft.fsa > SUP35_252seqs_mafft.fsa
```

In [97]:

```
#все успешно добавилось
! awk '/^>/ { count++ } END { print count }' SUP35_252seqs_muscle.fsa
! awk '/^>/ { count++ } END { print count }' SUP35_252seqs_mafft.fsa
```

252
252